# Adding a Custom System Call to the Linux Kernel

*Example: hello_syscall()*

This guide walks through the complete process of adding a new system call to the Linux kernel: writing the syscall, building it into the kernel, installing and booting the new kernel, and calling the syscall from a userspace program.

## Step 1 — Go to the kernel source directory

```
cd <path-to-kernel-source>
```

## Step 2 — Write the syscall implementation

```
nano kernel/hello_syscall.c
```

```
#include <linux/kernel.h>
#include <linux/syscalls.h>

SYSCALL_DEFINE0(hello_syscall)
{
    pr_info("Hello from my custom syscall!\n");
    return 123;
}
```

SYSCALL_DEFINE0 generates a zero-argument syscall entry point. pr_info() writes a message to the kernel log.

## Step 3 — Register the file with the build system

```
echo "obj-y += hello_syscall.o" >> kernel/Makefile
```

This tells the kernel build system to compile hello_syscall.c and link it into the kernel. A new file that isn't listed here will never be compiled.

Check the lines around the insertion point to make sure it wasn't added inside an ifdef/ifeq/ifneq block — if it is, that block's condition may cause it to be silently skipped during the build.

## Step 4 — Add an entry to the syscall table

```
nano arch/x86/entry/syscalls/syscall_64.tbl
```

Add a new line after the last existing entry, assigning the next free number:

```
471    common    hello_syscall        sys_hello_syscall
```

## Step 5 — Declare the function prototype

```
nano include/linux/syscalls.h
```

Add this line near the other syscall prototypes:

```
asmlinkage long sys_hello_syscall(void);
```

## Step 6 — Sync the kernel configuration

```
make olddefconfig
```

## Step 7 — Build the kernel

```
make -j$(nproc)
```

## Step 8 — Install kernel modules

```
sudo make modules_install -j$(nproc)
```

## Step 9 — Install the kernel image

```
sudo make install
```

## Step 10 — Update the bootloader

```
sudo update-grub
```

## Step 11 — Reboot into the new kernel

```
sudo reboot
```

## Step 12 — Confirm the running kernel

```
uname -r
```

## Step 13 — Write a test program

```
nano test_syscall.c
```

```
#include <stdio.h>
#include <sys/syscall.h>
#include <unistd.h>

#define __NR_hello_syscall 471

int main(void)
{
    long ret = syscall(__NR_hello_syscall);
    printf("syscall returned: %ld\n", ret);
    return 0;
}
```

## Step 14 — Compile and run

```
gcc test_syscall.c -o test_syscall
./test_syscall
```

Expected output:

```
syscall returned: 123
```

## Step 15 — Check the kernel log

```
sudo dmesg | tail -5
```

Expected line in the output:

```
Hello from my custom syscall!
```